

实验标题: Lab1: DataLab

姓名: 叶宇辰

学号: 25803050081

一, 在终端中执行 `./check_ops.py bits.c` 后的截图:

```
● yeyuchen@CHINAMI-N3RLDTE:~/projects/Lab1/Lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 9/20 operators
swapNibblePairs: 11/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 2/16 operators
roundEvenPow2: 23/24 operators
midpointTowardFirst: 21/32 operators
isBetweenEitherOrder: 37/48 operators
mul5Sat: 28/30 operators
classifyAdd3: 37/52 operators
floatScaleThreeHalves: 41/60 operators
floatRoundEven: 34/65 operators
float_i2f: 31/40 operators
bitCount: 29/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

二, 在终端中执行 `./btest` 后的截图:

```
● yeyuchen@CHINAMI-N3RLDTE:~/projects/Lab1/Lab1$ ./btest
Score  Rating  Errors  Function
1      1        0      signMask
2      2        0      bitXor
3      3        0      negativePart
4      4        0      copyByteWithin
4      4        0      logicalShift
4      4        0      swapNibblePairs
4      4        0      secondLowestZeroBit
5      5        0      oddParity
5      5        0      rotateRightBits
5      5        0      roundEvenPow2
5      5        0      midpointTowardFirst
7      7        0      isBetweenEitherOrder
7      7        0      mul5Sat
7      7        0      classifyAdd3
7      7        0      floatScaleThreeHalves
10     10        0      floatRoundEven
10     10        0      float_i2f
10     10        0      bitCount
10     10        0      bitReverse
Total points: 110/110
```

三，实现每个函数的思路：

P1 signMask：要求返回只有最高位为 1，其余全 0 的掩码，直接对 1 左移 31 位。

P2 bitXor：要求实现异或，即两位至少有一个为 1（即 $x|y$ ），且两位不能同时为 1（即 $\sim(x \& y)$ ）。

P3 negativePart：要求 $x < 0$ 时返回 $-x$ ，否则返回 0。对 x 右移 31 位看符号位，为 1（即 $x < 0$ ）时，该结果为全 1，将 $\sim x + 1$ 与该结果相与即得到 $-x$ ；为 0 时，该结果为全 0，相与即得到 0，符合要求。

P4 copyByteWithin：要求把 x 的 src 字节拷贝到 dst 字节，其余字节不变。先用移位和 0xFF 掩码取出 src 字节，再用掩码把 dst 字节清零，最后把取出的字节移到 dst 位置相或，得到结果。

P5 logicalShift：要求对 x 做逻辑右移 n 位（高位补 0）。先做算术右移得到结果，再构造一个“高 n 位为 0、其余位为 1”的掩码与之相与，把符号扩展出来的高位清零即可。位移采取先移 $31-n$ 位再移 1 位拼出掩码，避免 $n=0$ 时出现位移 32 的未定义行为。

P6 swapNibblePairs：要求把每个字节内部的高 4 位和低 4 位互换。先构造 0x0F0F0F0F 掩码，分别用它取出每个字节的低 4 位和高 4 位；再把低 4 位左移 4 位、高 4 位右移 4 位，最后相或拼合得到结果。

P7 secondLowestZeroBit：要求找 x 第二低位的 0。找最低位的 1 的公式为 $v \& (\sim v + 1)$ 。

（*参考了 ai）要找 x 最低位的 0，只需令 $v = \sim x$ ，问题即转化为找 $\sim x$ 的最低位 1。先用它找到第一个 0 位，并用或运算把它填成 1，再对结果重复一次同样的操作，找到的就是右起第二个 0 位。

P8 oddParity：要求根据 32 位中 1 的个数奇偶性返回结果（偶数个 1 返回 1）。利用异或就是模 2 加法的性质，采用分治折叠：依次把高 16 位与低 16 位异或、再折半为 8 位、4 位、2 位、1 位，最后最低位就成了所有 32 位的异或结果，对它取反即得到所需奇偶位。

P9 rotateRightBits：要求把 x 循环右移 n 位（低 n 位移到高 n 位，其余位右移）。先把 n 对 32 取模；然后分两部分拼：用算术右移加掩码得到高 $(32-n)$ 位下移后的部分（清掉符号扩展位），用左移把低 n 位搬到高位位置，最后两者相或即完成循环移位。掩码构造与 P5 逻辑移位同理。

P10 roundEvenPow2：要求把非负 x 就近舍入到 2^n 的倍数，恰好中间时，选商为偶数的那个。先取出低 n 位 $rest$ ，可以利用是否产生进位判断 $rest \geq 2^{(n-1)}$ ；当 $rest$ 恰好等于 $2^{(n-1)}$ 时，要看当前商 $x >> n$ 的奇偶，为偶数就向下取，奇数就向上取。最后用 $\sim mask$ 把低 n 位清零得到结果。

P11 midpointTowardFirst：要求无溢出地返回 $(x+y)/2$ ，恰好半整数时向 x 方向取整。先用公式 $(x \& y) + ((x \wedge y) >> 1)$ 无溢出地算出 $(x+y)/2$ 向下取整（利用 $x+y = (x \wedge y) + 2(x \& y)$ 的位恒等式）（*参考了 ai）；再用 $(x \wedge y) \& 1$ 判断 $x+y$ 是否为奇数。若为奇数，需决定是否向上补 1： $x \geq y$ 时向上取整， $x < y$ 时不变。比较 x 与 y 时，同号可直接用 $x-y$ 的符号位判断；异号则不能直接相减（可能溢出），改为看 x 的符号， x 非负则 $x > y$ ， x 为负则 $x < y$ 。最后把“需要补 1”与“是否半整数”相与，加到 mid 上得到结果。

P12 isBetweenEitherOrder：要求判断 x 是否落在 a 、 b 构成的闭区间内。先用与 P11 相同的无溢出比较法分别算出 $x \geq a$ 和 $x \geq b$ ；再用 $!ax$ 、 $!bx$ 判断 x 是否等于端点。最后“在区间内”等价于 $(x \geq a \text{ 且 } x \leq b)$ 或 $(x \geq b \text{ 且 } x \leq a)$ ，其中 $x \leq b$ 写成 $x < b$ 或 $x == b$ （即 $!xb | eqb$ ），两端取或。

P13 mul5Sat：要求返回 $x*5$ ，溢出时按方向饱和到 INT_MAX 或 INT_MIN 。

1.先算原始结果： $t = x << 2$ 得 $4x$ ，再加 x 得 $5x$ （记为 s ），这一步允许溢出，溢出问题后面单独处理。

2.判断是否溢出：大值粗判 big：当 $|x| \geq 2^{29}$ 时乘 5 必然溢出。正数用 " $x \geq 0$ 且第 29 位为 1" 判断；负数用 $x + 2^{29}$ 的符号位判断。

符号突变判：正常情况下 x 与 $5x$ 同号，若 s 与 x 符号位不同，说明发生了回绕溢出。需要两道溢出检测，是因为像 $x=0x40000000$ 这类值， $5x$ 回绕后恰好还是正数，符号突变检测不到，必须靠 big 兜住。（*参考了 ai）

3.确定饱和值 sat: $x \geq 0$ 溢出应饱和到 INT_MAX, $x < 0$ 应饱和到 INT_MIN。利用 $x >> 31$ 得到全 0 / 全 1 掩码，与 $\sim(1 < 31)$ (即 INT_MAX) 异或即可在两个饱和值间切换。

4.把 ovf 转成全 1 / 全 0 掩码 m ，用 $s \wedge (m \& (s \wedge \text{sat}))$ —— m 全 1 时返回 sat，全 0 时返回 s ，避免分支语句。

P14 classifyAdd3: 要求判断 $x+y+z$ 超出 INT_MAX/INT_MIN 的方向 (+1/-1/0)。

思路是分步累加、记录溢出贡献：先算 $x+y$ 得 s_1 ，再算 s_1+z 得 s_2 。每次两数相加时，溢出有两种方向：

1.两正数相加结果变负 $\rightarrow$ 正溢出，贡献 +1；

2.两负数相加结果变正 $\rightarrow$ 负溢出，贡献 -1。

把 $x+y$ 的溢出贡献 k_1 和 s_1+z 的溢出贡献 k_2 加起来，就是最终分类：因为三个 32 位数之和最多回绕一圈，所以总和为 +1 表示真和超过 INT_MAX，-1 表示低于 INT_MIN，0 表示在范围内。

P15 floatScaleThreeHalves: 要求按位实现单精度浮点数乘 $3/2$ ，用就近偶数舍入，NaN 原样返回。

先拆出符号位、阶码 exp、尾数 frac。

考虑特殊情况：

1.exp == 0xFF (NaN / 无穷)：直接返回原数。

2.exp == 0 (非规格化)：尾数本身没有隐式前导 1，直接对 frac 做 $\text{frac} + \text{frac}/2$ (即乘 1.5)，按就近偶数舍入 (最低位为 1 且结果为奇数时加 1)，再拼回符号位即可 —— 若结果进位到 bit 23，位布局自然变成规格化数 (阶码为 1)，无需特判。

一般情况 ($1 \leq \text{exp} \leq 254$)：

还原出 24 位有效数 $m = 0x800000 \mid \text{frac}$ (带隐式前导 1)，乘 1.5 即 $t = m + (m >> 1)$ ，同时记下 m 的最低位 rbit 作为舍入的粘滞位。

若 $t < 2^{24}$ (没溢出)：阶码不变。先对 t 做就近偶数舍入 (有余数 0.5 且 t 为奇数时加 1)；若舍入后恰好等于 2^{24} ，则需阶码 +1 (exp 为 254 时变为无穷大)。最后把结果减去隐式前导 1 拼回。

若 $t \geq 2^{24}$ (溢出需重新规格化)：阶码 +1，尾数除以 2；同样按就近偶数舍入 (看被丢掉的最底位和粘滞位)，exp 为 254 时结果为无穷大。

P16 floatRoundEven: 要求把浮点数按就近偶数舍入到最近整数。NaN / 无穷原样返回。

先拆出符号位、阶码 exp、尾数 frac。

考虑特殊情况：

exp == 0xFF: NaN / 无穷，直接返回。

exp >= 150: 即阶码减 127 ≥ 23 ，有效数的小数部分已经全部低于二进制小数点，本身就是整数，无需舍入，直接返回。

exp <= 125: 即数值绝对值 $\leq$ 约 0.25，舍入到最近整数必为 0，保留符号位返回 ± 0 。

exp == 126: 数值在 [0.5, 1.0) 之间。frac == 0 时正好 0.5，半整数就近偶取 0；否则 > 0.5 ，取 1.0。

一般情况 ($127 \leq \text{exp} \leq 149$)：

还原 24 位有效数 $m = 0x800000 \mid \text{frac}$ ，设 $E = \text{exp} - 127$ ，则数值为 $m \cdot 2^{-(E-23)}$ ，需要

右移 $\text{shift} = 23 - E = 150 - \text{exp}$ 位得到整数部分。

就近偶数舍入：在移位前给 m 加上偏移量 $(1 < (\text{shift} - 1)) - 1 + (q \& 1)$ ——其中 $(1 < (\text{shift} - 1)) - 1$ 保证超过一半就进位， $q \& 1$ 保证正好一半时向偶数看齐；加完再右移 shift 位得到舍入后的整数 N 。

把整数 N 编码回浮点：若 $N == 2^{(E+1)}$ ，说明舍入进位到了下一个 2 的幂，阶码 +1、尾数清零；否则 N 落在 $[2^E, 2^{(E+1)})$ 内，把 $N - 2^E$ 左移回 23 位尾数位置即可。

P17 float_i2f：要求把有符号整数 x 转换成单精度浮点的位表示。

整体流程：

1. 取符号和绝对值：符号位直接取 x 的最高位；若为负，用无符号运算 $0u - ux$ 求绝对值（兼容 `INT_MIN`）。 $x=0$ 直接返回 0。

2. 确定阶码：循环右移统计绝对值的最高位所在位置 n （即 $|x|$ 落在 $[2^n, 2^{(n+1)})$ ），则偏置阶码 $e = n + 127$ 。

3. 处理尾数：

若 $n \leq 23$ （有效位数 ≤ 24 位，可精确表示）：把绝对值左移 $(23 - n)$ 位，使最高位对齐到第 23 位，再用 `0x7FFFFFF` 掩码剥掉隐含前导 1，即得尾数。

若 $n > 23$ （位数过多需截断）：右移 $(n - 23)$ 位得到高位部分（含隐含 1），同时用掩码取出被丢弃的低位 `discard`。按就近偶数舍入：被丢弃部分大于一半则进 1；正好等于一半时，若当前尾数最低位为 1 则进 1（凑成偶数）。若舍入后尾数溢出到 `0x1000000`，说明进位到了新的 2 的幂，阶码 +1、尾数清零。最后同样剥掉隐含前导 1。

4. 拼合：符号位 | 阶码 $<< 23$ | 尾数，即得浮点表示。

P18 bitCount：用分治并行统计 1 的个数。先构造掩码 `0x55555555`、`0x33333333`、`0x0F0F0F0F`。第一步用 $x - ((x > 1) \& 0x55555555)$ 把每两位中 1 的个数算到这两位里；第二步按 2 位组两两相加到 4 位组；第三步按 4 位组相加到字节；再依次把字节间、16 位半区间相加，最后取低 6 位即为总 1 的个数。

P19 bitReverse：分治逐块交换。先用掩码把高低 16 位互换；再依次把相邻字节、相邻 4 位、相邻 2 位、相邻 1 位两两交换。每一步用对应掩码（`0xFFFF`、`0xFF00FF`、`0x0F0F0F0F`、`0x33333333`、`0x55555555`）取出两块，移位后相或拼回，逐步缩小交换粒度，最终得到 32 位反转结果。值得注意的是构造掩码须利用上一级掩码移位得到，否则容易超操作数。

P7: "（位运算技巧参考自 Hacker's Delight 第 2 章 / Anderson 《Bit Twiddling Hacks》）"

P11: "（无溢出平均公式参考自 Hacker's Delight §2.5；恒等式见 CS:APP 2.1.5）"

P13: "（溢出判定法则见 CS:APP §2.3.2；同号回绕边界为自行分析）"